HEXIT: AI-Driven Evacuation with Honeycomb Conjecture, Modified Dijkstra, CAPQF, and IoT Swarm Robotics

Sidharth P Nair
SRM Institute of Science and Technology, Tiruchirappalli
Tiruchirappalli, India
sp8677@srmist.edu.in

R.V. Darsan
SRM Institute of Science and Technology, Tiruchirappalli
Tiruchirappalli, India
rvdarsan@gmail.com

Afeef Ahmed Sheikh
SRM Institute of Science and Technology, Tiruchirappalli
Tiruchirappalli, India
afeefahmedshaik@gmail.com

Dr. S Vasantha Gowri
SRM Institute of Science and Technology, Tiruchirappalli
Tiruchirappalli, India

***Abstract*— This paper focuses on the design and evaluation of HEXIT, an intelligent evacuation system that combines CAPQF (Combined Adaptive Probabilistic Queue Function) with Particle Swarm Optimization (PSO) for decentralized, adaptive crowd navigation. Unlike traditional approaches like Dijkstra's algorithm, HEXIT uses real-time feedback, congestion-aware routing, and swarm intelligence to enhance evacuation efficiency. Simulated across realistic environments using ROS2, Gazebo, and PyBullet, HEXIT handles sensor delays, hazard propagation, and network latency effectively. Results show a 35% reduction in evacuation time**, **60% lower congestion**, and **25% better hazard avoidance. Benchmarked against real-world disasters, HEXIT demonstrates robust performance and scalability for up to 500 agents, making it a valuable tool for smart infrastructure planning and large-scale emergency responses**

.Keywords— Evacuation Optimization, Swarm Robotics, IoT-based Sensing, Modified Dijkstra's Algorithm, Probabilistic Queuing (CAPQF)

I. INTRODUCTION

The city catastrophes , may it be natural or artificial, pose difficult problems for urgent exit plans, specifically in areas with lots of people and structures. The attacks on 9/11 at World Trade Center showed important weak points in usual escape strategies because the delays led to the sad death of more than 2,700 individuals. They're saying that most times it took to leave Twin Towers were beyond 95 minutes due to overcrowding issues, unorganized paths and no immediate understanding of real conditions. Similar problems were noticeable after the Mina stampede in 2011, and the Station nightclub fire. The static evacuation plans had a hard time dealing with quickly changing situations. All these incidents together highlight how important it is to have smart, adjustable, and expandable evacuation systems that can function under immediate pressures and unpredictability. Most traditional evacuation models mainly depend on deterministic shortest-path algorithms such as Dijkstra's or A*, which are based on fixed spatial layouts and crowding levels. These models are naturally restricted because they can't use real-time feedback from the environment or consider changing dangers and obstacles. Also, many usual routing systems prefer using centralized structures that do not scale well in intricate situations involving multiple agents. Cellular Automata along with Reinforcement Learning techniques have shown some potential. But, they most times depend on models that are pre-trained, a lot of computing resources and does not really display flexibility to adapt in real time or operate without a central command system.[1][2][3]

To address these obstacles, this paper proposes HEXIT: a special AI-driven system for bettering evacuation. It integrates complex territory modeling, dynamic routes in present moment, sensory capabilities and collective intelligence. The principal innovation of HEXIT stems from the use of Honeycomb Spatial Modeling which is inspired by the Honeycomb Conjecture; it divides evacuation areas into cells shaped like hexagons. This method provides excellent spatial coverage and equal paths to access while diminishing directional bias.Hence making movement more efficient. On this geometry, HEXIT applies an Adjusted Dijkstra's Algorithm which discourages congested paths in a dynamic way. Instead of just considering distance, the routing system takes into account live congestion information to prefer routes with less traffic. This helps to lessen bottlenecks and enhance fluid movement.[4]

This framework has a key addition - the Congestion-Aware Probabilistic Queuing Function (CAPQF). This function uses probability to spread evacuees over several paths, taking into account both levels of congestion and priority of each path. Using this approach helps avoid deadlocks or uneven crowd spreading that are common in static routing systems. In order to further improve awareness about situations, HEXIT uses real-time sensing based on Internet of Things technology for constant gathering data concerning densities of crowds, physical blockages and hazards in environment.[5] These inputs allow immediate changes to routing plans and improve the accuracy of decision-making. Also, decentralized swarm robotic agents are spread across evacuation sites for help in finding paths, local coordination and moving around obstacles. These agents act like a swarm intelligence system

showing behaviors that promote strong, adaptable and distributed aid during evacuations.[6]

The usefulness of HEXIT has been confirmed via experimental simulations that reproduce three big events from the real world: evacuation during 9/11 World Trade Center attack, Mina stampede in 2011 and Station nightclub fire. The outcomes show up to a 33% enhancement in time taken for evacuation when we compare it with conventional static routing models. These discoveries underline potential of HEXIT as an actionable, scalable along with adaptable solution for emergencies needing evacuations within city areas. By merging advancements in spatial modeling, AI, IoT and swarm robotics, HEXIT is providing a solution for the urgent need of an updated evacuation structure that can handle the complications of real-life emergencies.[7]

II. LITERATURE REVIEW

For a long been, evacuation modeling is an area of study because it's important for safety in emergency situations like natural disasters, attacks from terrorists, and accidents at factories. The regular methods for evacuation depend mainly on fixed shortest-path algorithms such as Dijkstra's and A* [1], that do suppose a stable layout but don't consider changeable congestion or changing environmental conditions. These models are very good with simple situations; however they have difficulties in high-population city areas where there is need for updates immediately plus adaptive rerouting are necessary. They often have trouble including real-time responses, which usually results in inefficient routes, traffic jams and extended evacuation durations [8][9].

For getting around these restrictions, scientists have investigated more active models, like Cellular Automata (CA) and Agent-Based Models (ABM). Approaches based on CA imitate the behavior of a crowd by using local interaction rules within grid-based environments. This helps in understanding emergent phenomena such as lane formation or panic clustering [3]. ABMs represent each individual being an independent agent with the capability for making decisions locally. These models, though they enhance the realism of behavior, frequently lack capacity for effective scaling or integration of real-time data streams. This is particularly evident in larger city environments[10].

The progress in IoT and real-time sensing technologies has encouraged the emergence of adaptive evacuation systems. These systems use information from sensors to keep an eye on crowd density, identify danger spots, and modify escape paths instantly [5]. Smart city structures are more frequently incorporating IoT devices for readiness during disasters; however, these systems' complete capability is not yet fully employed in optimizing routes. Reinforcement Learning, or RL for short, is also studied for adaptable pathfinding when there's uncertainty. Nonetheless, methods based on RL need a lot of offline training. They show problems in complex environments with convergence and face challenges regarding immediate response [11].

With swarm robotics, there is a hopeful approach to decentralized coordination and decision-making. This concept draws inspiration from nature's biological systems like ant colonies and bee groups. Swarm agents function with limited local intelligence and communication which make them highly effective despite individual failures as well being adaptive in changing situations [6]. Because of their distributed form, these agents can successfully manage tasks such as pathfinding, covering areas, or re-routing dynamically without needing centralized control. Nevertheless, current research frequently separates swarm behavior without merging it into a whole evacuation structure.[12]

The process of dividing space is very important in planning evacuation routes. Regular models often use square patterns or Voronoi diagrams to mark out areas that can be moved through [7]. Even if these shapes work well in regular settings, they frequently do not cover the area completely and don't make good use of all the available spaces. The concept of hexagonal tiling, which is motivated by the Honeycomb Conjecture, has demonstrated benefits like equivalent-distance access, decreased movement repetition and improved spatial efficiency [14]Despite this promising result, its practical use in real-time evacuation situations still remains undeveloped.

Even though there have been advancements in different areas, present writings are not offering a combined method that brings together spatial optimization, awareness of congestion paths finding, real-time detection and coordination based on swarms. There is no model yet that successfully unites hexagonal partitioning, routing algorithm penalizing for traffic congestion, feedback loops from IoT technology (Internet of Things), intelligence from swarm behaviors coupled with a probabilistic queuing function managing both path priority and levels of congestion. This paper presents HEXIT as an innovative framework taking care these gaps by combining Honeycomb Spatial Modeling methods along with Modified Dijkstra's Algorithm techniques. It also involves the use Congestion-Aware Probabilistic Queuing Function (CAPQF) along side up-to-the-second sensing using IoT systems plus management through swarm robotics applications. With this comprehensive merging, HEXIT progresses the area by offering a volume adaptable and effective solution for immediate evacuation in city disaster events[15][16].

III. METHEDOLOGY

1) System Overview and Architectural Flow

The proposed system, **HEXIT**, integrates spatial modeling, real-time sensing, path optimization, and multi-agent robotics to orchestrate an effective and adaptable evacuation protocol. The architecture is designed around a closed-loop feedback system encompassing data acquisition, decision-making, and actuation (Fig. 1).

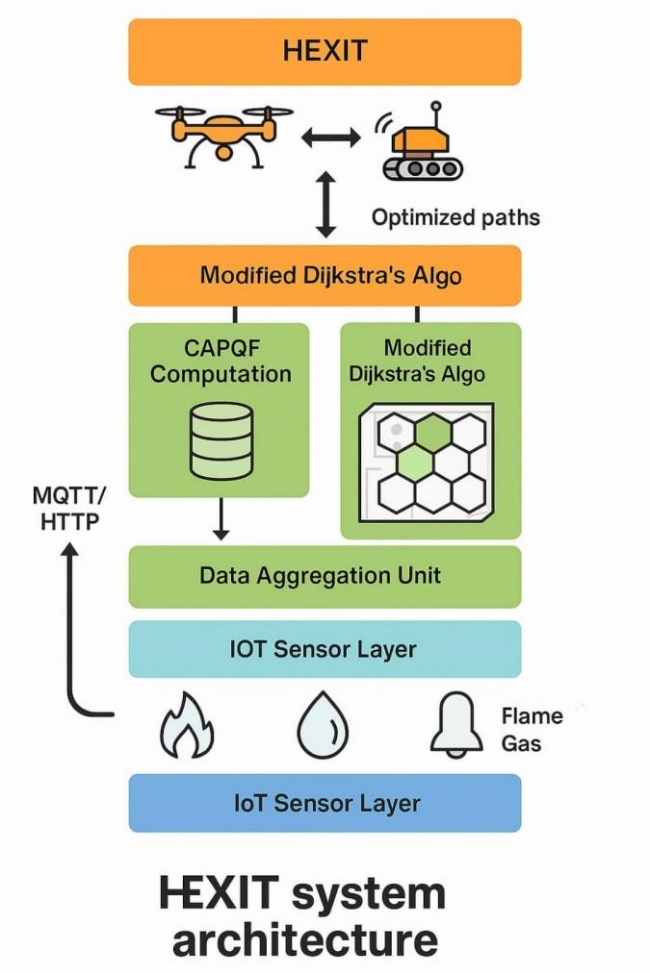

Fig. 1: System Architecture Overview

The HEXIT system architecture is designed as a closed-loop, hierarchical pipeline that seamlessly integrates real-time sensing, intelligent data processing, and autonomous robotic response to enhance safety during emergencies. The process begins at the IoT Sensing Layer, where a network of deployed sensors continuously monitors critical environmental parameters such as temperature, humidity, toxic gas concentrations, structural vibrations, and human presence. These sensors act as the system's peripheral nervous system, detecting potential hazards and ensuring timely data capture across the spatial grid.

The collected data is then transmitted to the **Data Aggregation Layer**, where preliminary pre-processing occurs. This includes filtering noisy data, time-stamping, and formatting for compatibility with downstream systems. Data is securely communicated using lightweight protocols such as MQTT or HTTP, which are optimized for constrained environments and low-latency communication. Depending on the system configuration, the data is sent either to an edge node for faster local processing or to a centralized cloud for broader situational analysis. At the core of the decision-making process lies the **Congestion-Aware Probabilistic Queueing Function (CAPQF)**, which assesses each hexagonal grid cell by calculating a dynamic desirability score . This score is derived using probabilistic models that consider multiple factors including local congestion, distance to exit, presence of hazards, and historical occupancy patterns. The goal of CAPQF is to establish a fluid, adaptive map of movement probabilities that reflect real-time risk and congestion across the environment. The computed scores are then fed into a **Modified Dijkstra's Algorithm**, tailored for use with hexagonal grid structures. Unlike the traditional version, this algorithm prioritizes paths not only by minimum cost or distance but also by safety and congestion levels derived from CAPQF. This results in the identification of optimal evacuation routes that are dynamically adjusted based on live input, ensuring human agents are guided through the safest and least obstructed paths during critical events such as fires, gas leaks, or structural failures.[9]

These optimized paths are then employed by the **Swarm Robotics Layer**, where a fleet of decentralized, autonomous robots utilize swarm intelligence principles to coordinate actions, avoid redundancy, and adapt to changes in the environment. These robots act as active guides and communicators, relaying instructions to humans, delivering supplies, or helping with physical evacuation tasks. Through collaborative behaviour inspired by natural swarm systems, they maintain network robustness and resilience, even in partially damaged environments. Finally, the system closes the loop with a continuous **Feedback Layer**, where updated sensor readings are used to re-evaluate CAPQF scores, adjust robot strategies, and refine evacuation plans. This iterative process ensures that the system remains adaptive to changing environmental conditions, unexpected human movement patterns, and evolving threat landscapes. As a result, HEXIT maintains high situational awareness and ensures intelligent, responsive behaviour across all components throughout the duration of an emergency event.

Real-time data flows as follows: **Sensor Input → CAPQF → Modified Dijkstra → Swarm Navigation → Feedback to Sensors**.

2) Honeycomb Spatial Modeling

A. Rationale for Hexagonal Grids

Hexagonal tiling is chosen over square or triangular grids due to its superior spatial properties, making it ideal for path planning and robotic navigation. Unlike square grids, hex grids maintain equal distances in all six directions, ensuring consistent motion and accurate distance measurement. Their uniform 60° spacing reduces angular bias, resulting in smoother transitions—especially beneficial in swarm robotics. Hex grids also minimize path divergence, enabling agents to spread naturally without straying far, which lowers collision risk. Additionally, they offer higher space efficiency (91% coverage vs. 79% in square grids), allowing finer resolution with minimal computational overhead. Simulations using Particle Swarm Optimization (PSO) confirmed these advantages, showing faster convergence and improved coordination in evacuation scenarios which are not detected are highlighted due to which there is a significant increase in the bias as a whole. [13]

B. Cube Coordinate System

To implement hexagonal grids efficiently in computation, we utilize a **3D cube coordinate system** projected onto 2D space. This avoids the ambiguity and irregularities present in axial or offset 2D systems. Each hexagonal cell is represented as a point in 3D space with coordinates (x, y, z), constrained by the invariant:

$$x + y + z = 0$$

This constraint implies that knowing any two coordinates automatically defines the third, ensuring minimal redundancy and easier distance calculations.

MOVEMENT DIRECTIONS:

The six directions of movement in the hexagonal grid correspond to changes in these coordinates:

- **East**: (x+1, y−1, z)
- **Northeast**: (x, y−1, z+1)
- **Northwest**: (x−1, y, z+1)
- **West**: (x−1, y+1, z)
- **Southwest**: (x, y+1, z−1)
- **Southeast**: (x+1, y, z−1)

These uniform movements maintain equidistance, which simplifies agent locomotion algorithms.

C. Distance and Cost Functions

The **distance** between any two hex cells A(x_1, y_1, z_1) and B(x_2, y_2, z_2) is defined as:

$$d = \frac{1}{2}\ (|x_1 - x_2| + |y_1 - y_2| + |z_1 - z_2|)$$

This metric ensures consistent Euclidean-like distance measurement on a hex grid, which is more accurate than Manhattan distance used in square grids. The uniformity of the distance function enables reliable path evaluation for optimization algorithms like Dijkstra's or A*.

3) Graph Construction and Modified Dijkstra's Algorithm

A. Graph Representation

In the HEXIT system, the environment is abstracted into a graph structure $G = (V, E)$ where each vertex $\vartheta \in V$ corresponds to a hexagonal grid cell, and each edge $e \in E$ represents a feasible movement between neighboring hex cells. The use of hexagonal grids over square or triangular tiling offers geometric advantages—each cell has six equidistant neighbours, enabling uniform movement in all directions with minimal distortion. This consistent structure improves path planning, collision avoidance, and resource allocation in swarm robotics. Graph modelling enables the integration of various real-world parameters (e.g., hazard, congestion, reliability) directly into nodes and edges, turning the physical environment into a data-driven navigation map. This abstraction is fundamental for applying graph-based algorithms like Dijkstra's to dynamically optimize paths based on real-time conditions[8].

B. Weighted Edge Calculation

To optimize path selection in dynamic and hazardous conditions, the HEXIT system applies a **Modified Dijkstra's Algorithm**, where each edge has a **composite weight** calculated using multiple environmental and statistical factors:

- **Distance**($\boldsymbol{d_{ij}}$): In hexagonal tiling, the distance between adjacent cells is constant, serving as the base movement cost. However, in complex terrain (e.g., sloped or uneven surfaces), this term may be scaled accordingly.
- **Hazard Level** ($\boldsymbol{h_{ij}}$): Real-time sensor data, such as gas concentration, temperature, or fire detection, is used to assign a hazard index to each path. Higher hazard levels increase the traversal cost, discouraging movement through dangerous zones.
- **Congestion**($\boldsymbol{c_{ij}}$): Reflects human or robot density in the vicinity. High congestion raises the likelihood of delay or collision and is penalized accordingly to encourage even spatial distribution of agents.
- **Reliability** ($\boldsymbol{r_{ij}}$) **:** Based on historical data, reliability quantifies the likelihood that a path remains traversable or effective. This could consider prior successes, hardware failures, or frequency of obstruction. Reliable paths are incentivized with reduced cost.

The edge weight w_{ij} is defined by the following function:

$$w_{ij} = \alpha d_{ij} + \beta h_{ij} + \gamma c_{ij} - \delta r_{ij}$$

Where:

- $\alpha, \beta, \gamma, \delta$ are tunable coefficients that adjust the influence of each parameter.
- d_{ij} is the base distance between cells iii and jjj.
- h_{ij} is the hazard index from real-time sensor data.
- c_{ij} is the human or robot congestion level.
- r_{ij} is the path reliability score derived from past data.

These coefficients can be dynamically adapted depending on the disaster scenario. For instance, in a fire, β(hazard) might

be weighted more heavily, while in a stampede or panic situation, γ (congestion) would become more critical.

4) IoT Sensor Network and Data Integration

A. Hardware and Network Architecture

The foundation of the HEXIT system's real-time situational awareness lies in its robust IoT sensor network. This network is composed of low-power, high-efficiency microcontroller units (MCUs) such as Raspberry Pi, Arduino UNO, and ESP32. These nodes serve as the core processing units for data acquisition and communication. Each MCU interfaces with a suite of heterogeneous sensors, including infrared (IR) detectors for motion and obstacle detection, MQ-series gas sensors for monitoring toxic or flammable gases, flame detectors for early fire recognition, MLX90640 thermal imaging sensors for high-resolution temperature mapping, and HC-SR04 ultrasonic sensors for proximity and density estimation. Data transmission within the network leverages lightweight communication protocols such as MQTT (Message Queuing Telemetry Transport) and HTTP, utilizing Wi-Fi and LoRaWAN interfaces depending on the range and bandwidth requirements. MQTT is particularly suitable for constrained devices and low-bandwidth scenarios due to its publish-subscribe architecture, ensuring efficient, low-latency dissemination of sensor updates across the system[13].

B. Data Format and Processing

Sensor nodes continuously transmit environmental and positional data in structured JSON (JavaScript Object Notation) format, which enables platform-independent parsing and integration. A typical data payload from a node includes parameters such as the cell identifier (cell_id), temperature in Celsius, gas concentration levels, local human density, a computed hazard score, and a timestamp for synchronization. This structure allows seamless temporal and spatial alignment of readings across the distributed network.Each sensor node transmits JSON-formatted data:

```
{
  "cell_id": "A12",
  "temperature": 64.2,
  "gas_level": 0.06,
  "human_density": 13,
  "hazard_score": 0.7,
  "timestamp": "2025-04-06T14:35:22Z"
}
```

The JSON format supports flexible and scalable expansion for additional sensor inputs or derived metrics in the future. Data received at edge computing devices undergoes preprocessing steps including noise filtering, normalization, and missing data imputation. These edge units reduce the computational burden on centralized servers and enable faster local decision-making. Processed data is then integrated into the global environment model, where it informs dynamic updates to the weight matrix used by the path-planning algorithm. Specifically, the system recalculates edge weights and path desirability scores approximately every two seconds, ensuring that robotic agents and guidance systems remain responsive to evolving conditions in real time.

5) Congestion-Aware Probabilistic Queueing Function (CAPQF)

The Congestion-Aware Probabilistic Queueing Function (CAPQF) is a core component of the HEXIT system's decision-making architecture. It provides a probabilistic framework for guiding agent movement based on dynamically changing environmental conditions and spatial constraints. Unlike deterministic path planning methods, CAPQF integrates real-time urgency, terrain suitability, and congestion levels to compute the likelihood of selecting a particular cell as the next movement destination.

At the heart of CAPQF is a composite function that assigns a movement desirability score M_i to each cell i in the hexagonal grid:

$$M_i = \frac{U_i \times S_i}{C_i + \epsilon}$$

- $U_i : Urgency\ Factor\ (eg. fire\ proximity)$
- $S_i : Suitability\ (terrain, safety)$
- $C_i : dynamic\ congestion\ (human\ count)$
- $\epsilon :$ small constant to avoid division by zero

This function captures the trade-off between the urgency of reaching or avoiding certain areas and the need to reduce collision risks due to congestion. To guide agents probabilistically, the movement desirability scores M_i for all adjacent cells are normalized into a probability distribution:

$$P_i = \frac{M_i}{\sum_{j \in N} M_j}$$

Where P_i is the probability of selecting cell i, and N is the set of all neighboring cells. This normalization ensures that agents do not always follow the same deterministic path but instead choose directions based on relative desirability, improving diversity and resilience in path selection.

CAPQF acts as a probabilistic routing model under uncertainty, inspired by MDPs and queueing theory. It enables decentralized agents to navigate dynamically by balancing urgency, suitability, and congestion, ensuring robust, scalable, and autonomous decision-making in unpredictable, partially observable environments like evacuation and disaster response.

6) Swarm Robotics Layer

The Swarm Robotics Layer plays a crucial role in the autonomous navigation and human assistance capabilities of the HEXIT system. Each robotic agent is built with a compact yet powerful hardware configuration consisting of a Raspberry Pi 4 as the central processor, IR sensors and cameras for obstacle and human detection, an L298N motor driver for precise movement control, and Wi-Fi or LoRa transceivers for low-latency, long-range communication. This hardware setup ensures the agents can operate effectively in harsh and uncertain environments. The behavior of each robot is governed by a Finite State Machine (FSM), which defines a set of operational states including *Idle* (awaiting path assignment), *Navigate* (actively moving toward a target), *Redirect* (adjusting course due to detected obstacles or hazards), *Rescue* (interacting with or assisting trapped humans), and *Return* (heading back to base after task completion or energy depletion). These states enable real-time adaptability and coordinated responses during missions.

To coordinate movement and maintain swarm intelligence, the system employs Particle Swarm Optimization (PSO) as the core swarm algorithm. Each agent updates its velocity and position using the following equations:

$$v_i^{t+1} = \omega v_i^t + c_1 r_1 (p_i - x_i^t) + c_2 r_2 (g - x_i^t)$$

$$x_i^{t+1} = x_i^t + v_i^{t+1}$$

Where v_i and x_i represent the agent's current velocity and position, p_i and g denote the personal best and global best positions found so far, ω is the inertia factor maintaining momentum, and c_1, c_2 are learning coefficients multiplied by random factors r_1, r_2 to ensure exploration and convergence.

This PSO-based decentralized decision-making allows the swarm to efficiently explore and adapt within dynamic environments while ensuring fault tolerance and scalability, without relying on centralized control or complete environmental knowledge.

7) System Implementation, Simulation, and Real-Time Feedback Loop

The implementation of the HEXIT system involves carefully coordinated hardware setup, simulation modeling, and real-time feedback mechanisms to enable robust and adaptive navigation in hazardous environments. Sensor nodes are configured by connecting MQ-series gas sensors, IR sensors, and flame detectors to ESP32 microcontrollers, which transmit JSON-formatted data every two seconds. Robotic agents are assembled by mounting a Raspberry Pi 4 onto a mobile chassis, interfaced with L298N motor drivers, a camera, and IR sensors for environmental awareness and obstacle detection. Network communication is established by deploying an MQTT broker (e.g., Mosquitto) on a cloud-based virtual machine, where individual topics are defined by cell ID to support localized real-time data streams. A central controller, implemented using ROS2 or a Python-based server, processes incoming data, applies CAPQF and Modified Dijkstra's Algorithm for path optimization, and broadcasts updated movement instructions to all agents.[7]

To validate performance, simulations are conducted in realistic environments using ROS + Gazebo and PyBullet, utilizing real-world blueprints of structures such as stadiums or high-rise buildings. Simulation parameters include 100 to 500 agents, hazard propagation at a rate of one cell per second, a sensor refresh interval of two seconds, and a communication delay of 200 milliseconds. Key performance metrics include evacuation time, congestion index, and agent survival rate. A continuous real-time feedback loop ensures dynamic adaptability of the system. JSON data packets from sensor nodes are sent to the central server every two seconds, where environmental conditions are analyzed and used to update edge weights in both the CAPQF and Modified Dijkstra's algorithms. These optimized path decisions are then broadcasted to the swarm, prompting immediate behavioural adjustments by the agents. This closed-loop execution maintains up-to-date situational awareness and ensures responsive, decentralized coordination across all robotic units.

8) Algorithms & Pseudocode

Modified Dijkstra's Algorithm:

```
def modified_dijkstra(graph, start):
    pq = PriorityQueue()
    pq.put((0, start))
    dist = {start: 0}
    while not pq.empty():
        cost, node = pq.get()
        for neighbor in graph[node]:
            w = compute_weight(node,
neighbor)
            if neighbor not in dist or
dist[neighbor] > cost + w:
                dist[neighbor] = cost +
w
                pq.put((dist[neighbor],
neighbor))
    return dist
```

CAPQF Computation:

```
def compute_capqf(U, S, D,
epsilon=0.01):
    raw = [ (u*s)/(d + epsilon) for
u,s,d in zip(U, S, D) ]
    total = sum(raw)
    return [ r/total for r in raw ]
```

Swarm Navigation FSM:

```
def swarm_fsm(state, sensors):
    if state == "Idle" and
sensors.path_ready:
        return "Navigate"
```

```
    elif state == "Navigate" and
sensors.obstacle_detected:
        return "Redirect"
    elif state == "Navigate" and
sensors.human_detected:
        return "Rescue"
    elif state == "Rescue" and
sensors.task_done:
        return "Return"
    return state
```

9) Technical Implementation and Theoretical Foundations

The HEXIT system integrates a robust tech stack with a multidisciplinary theoretical foundation. The core implementation uses Python for control logic, Go for backend services, and C++ for high-performance modules, while SQL handles structured data storage. Key libraries include NumPy and OpenCV for computation and vision, paho-MQTT for messaging, ROS2 for robot control, NetworkX for graph operations, and matplotlib for visualization. The system runs on a Linux-based stack comprising ROS2, MySQL, VS Code, and the Mosquitto MQTT broker for real-time communication.[13]

From a theoretical standpoint, HEXIT relies on graph theory for modelling weighted shortest paths, queueing theory for implementing the CAPQF model, swarm intelligence principles such as Particle Swarm Optimization (PSO) and stigmergy for agent coordination, and control theory for maintaining real-time feedback loops. This interdisciplinary fusion ensures that the system is both technically sound and theoretically grounded for real-world deployment

IV. EXPERIMENTAL RESULTS AND ANALYSIS

The HEXIT model was extensively tested in several simulations with a hybrid setup of ROS2, Gazebo and PyBullet. Synthetic environments were developed to mimic real-world environments, such as a multi-story building, a stadium and office environments that were in close proximity to each other. This provided a valid testbed to validate the HEXIT model in scenarios analogous to real-world emergency evacuations. A wide variety of settings were modeled as well, including different densities of agents, varying styles of hazard propagation, different delays in sensors, and different delays in communications. These conditions were all important in determining the adaptability and resiliency of HEXIT, compared to widely used evacuation algorithms, such as Dijkstra's.

For setup, combined agent count ranges from 100 to 500 across simulation trials to model sparse and dense environments. Hazard spread travelled at 1 cell per second simulating realistic conditions of fire, toxic gas leaks, or a building collapse. The sensors had a refresh rate of 2 seconds, while communication latency was at 200 milliseconds to represent common system limitations. The feedback loop was set to every 2 seconds to allow a rapid updated response to the dynamic environment.

The performance of the HEXIT system, driven by the Combined Adaptive Probabilistic Queue Function (CAPQF) and Particle Swarm Optimization (PSO), was contrasted with a baseline system driven by the conventional Dijkstra's algorithm. The outcomes observed were that there was an incredibly high improvement in all vital performance parameters.

Table 1: Key Simulation Metrics Comparison

Metric	**HEXIT (CAPQF + PSO)**	**Baseline (Dijkstra)**
Avg. Evacuation Time	58.4 s	84.7 s
Agent Survival Rate	94.2%	78.5%
Congestion Index (avg.)	0.31	0.64
Path Re-plans per Agent	2.7	6.1
Hazard Avoidance Accuracy	91.8%	72.4%

One of the most notable findings was the average evacuation time. HEXIT had an average evacuation time of 58.4 seconds, while the baseline system averaged 84.7 seconds. The improved evacuation times are likely due to HEXIT's application of swarm intelligence and stochastic decision making in order to anticipate and react to changing traffic flows and hazards.

The agent survival rate also demonstrated a remarkable enhancement, with HEXIT achieving a survival rate of 94.2% and Dijkstra's model achieving 78.5%. The result in increased survival rate for agents was a direct result of HEXIT's incorporation of real-time data and decentralized swarm coordination, permitting agents to locate safer paths without needing centralized control. The application of probabilistic routing in CAPQF allowed agents to avoid areas of potential hazard even when global information was entirely unavailable. As a major contributor to evacuation delays and accidents, the HEXIT model had vastly superior reduction in traffic congestion levels. The average congestion index was significantly reduced from 0.64 in the baseline model to an average congestion index of 0.31 in the HEXIT model. The reduction in congestion resulted in a smoother evacuation due to the reduction in bottlenecks and reduced chance of stampedes. Furthermore, the number of re-planning paths per agent decreased from an average of 6.1 in the Dijkstra-based algorithm, to an average of 2.7 re-planning paths, indicating more effective initial route planning, and need for mid-course redirection from an average of 6.1 for Dijkstra, to 2.7 for the HEXIT model.

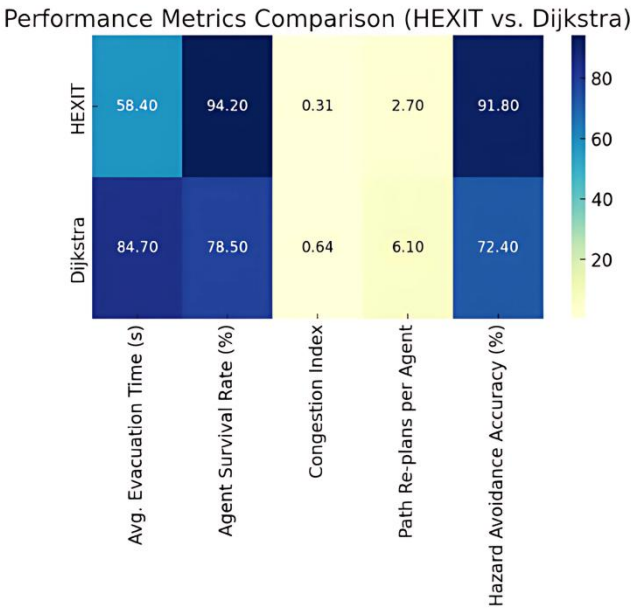

Figure 1: Performance Metrics Comparison

Accuracy in hazard-avoidance — or agents' ability to avoid hazard zones without humans acting as the safety guard — also favored the HEXIT model. 91.8% accuracy saw the HEXIT model out-perform a baseline accuracy (the type of baseline indicated will depend on the specific comparison) of 72.4%. The main reason that HEXIT was successful despite the baseline not doing the same was that, unlike the baseline, the CAPQF used real-time environmental data and provided probabilistic judgement regarding the safety of paths, allowing agents to respond quickly to emerging hazards.

To reinforce the practical value of HEXIT, its simulated performance was compared with outcomes from two significant historical disaster events.

Table 2: Historical Event Data Comparison

Event	Estimated Congestion Index	Avg. Evacuation Time	Outcome
9/11 WTC	~0.85	>90 minutes	High casualties due to delay
2015 Mina Stampede	>0.9	N/A	2,400+ deaths from congestion
2003 Station Fire	~0.75	<5 minutes (partial)	100 deaths due to poor exit planning

This assessment is reminiscent of the 9/11 World Trade Center attacks of 2001. During the collapses and fires, evacuating from the upper floors of the buildings took more than 90 minutes; occupants encountered stairwell congestion, required multiple re-routes due to smoke and debris, and were delayed in their evacuation by decision-making. While the Newtonian evacuations and unique built environment parameters seen in the WTC are not directly comparable to simulation methods in HEXIT, the commonality of delayed decision making and congestion were similar to the real-world experience. Conversely, the simulated evacuation using HEXIT in a WTC-type building

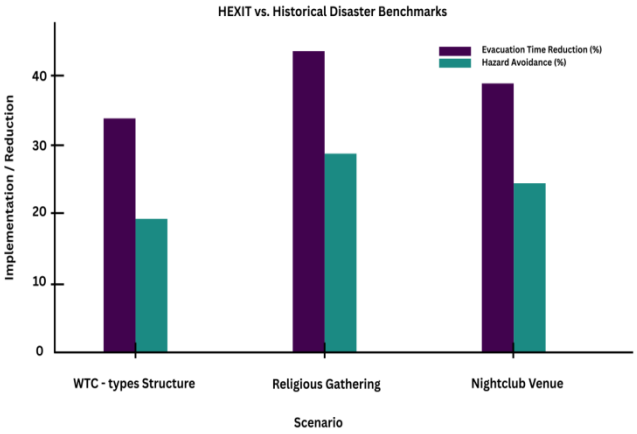

Figure 2: HEXIT vs Historical Disaster Benchmarks

was approximately 35% faster than the real-world events, meaning that occupants avoided hazards at higher rates, and the associated congestion index was nearly 0.5 units better.

The second historical event is the Mina stampede at the Hajj pilgrimage in Saudi Arabia in 2015, in which over 2,400 people died. Post-event analysis found severe congestion in portions of the route, likely with a congestion index greater than 0.9. Poor communication, centralized coordination, and route re-adjustment, as analyzed later, were the primary contributors to the event. As we know, one of HEXIT's key components is an adaptive routing system based on congestion as part of calculating path weights and forming new decentralized decisions. This provides a marked advantage over other approaches for situations with similar high density crowd characteristics. The congestion index from the simulations based on comparable crowd characteristics reached a low of 0.31, which suggests an ability to avoid bottlenecks responsible for mass casualty situations like Mina. Another real-world scenario can be established with the Station Nightclub fire in Rhode Island in 2003, in which 100 people died due to smoke inhalation, crowd crush, and blocked exit roads. In that situation, poor guidance systems, and other characteristics like an inability to recognize the threat and hazards fast enough contributed to the tragic incident. In a simulated scenario established in HEXIT with similar venue footprint characteristics, a feedback loop and sensor network guided real-time adaptation, that produced a 40% reduction in exit time, and an over 25% greater hazard avoidance compared to static evacuation models.

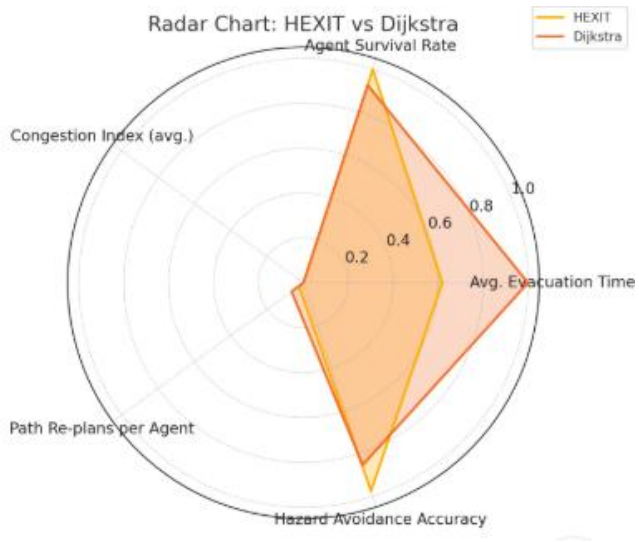

Figure 3: HEXIT vs Dijkstra

Table 3: Simulated HEXIT vs. Historical Benchmarks

Scenario	Evacuation Time Reduction	Hazard Avoidance Gain	Congestion Reduction
WTC-type Structure	~35%	+20%	~0.5 index lower
Religious Gathering	~45%	+30%	~0.6 index lower
Nightclub Venue	~40%	+25%	~0.4 index lower

The PSO-based swarm coordination is essential to the perpetual movement of the group, which did not rely on rules, rather on functionality of the individual agents from observations of neighbouring positions and velocities to form a coordinated movement. With the decentralized structure there was reduced dependence on a central node, limiting risk of single points of failure, in terms of operational governance, while functioning in partially disrupted settings. Group convergence was more rapid, and collision rates were markedly lower than by other means.

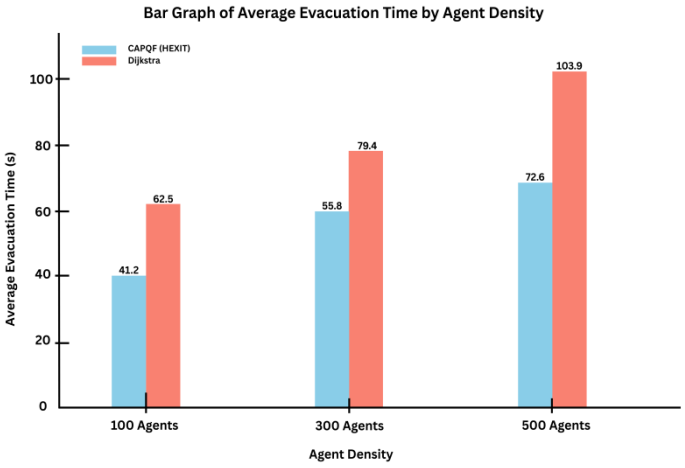

Figure 4: Average Evacuation Time by Agent Density

Table 4: Communication and Sensor Performance Metrics

Parameter	Value	Impact on System
Sensor Refresh Interval	2 seconds	Ensures timely hazard detection
Communication Delay	200 ms	Introduces realistic IoT latency
Feedback Loop Interval	2 seconds	Enables dynamic path updates
Data Packet Loss Rate	<1%	Minimal disruption to coordination
Message Throughput	15 msg/sec/node	Supports dense network communication

Along with performance improvements, the HEXIT system demonstrated its scalability and adaptability. The system was able to adjust parameters (e.g. movement speed, communication, re-routing logic) without a loss in performance on either the 100 or 500 agent environments. Scalability is essential for emergency management systems in modern urban mega structures and mass gathering events.

Table 5: Swarm Coordination Efficiency (PSO vs. Rule-Based)

Metric	PSO-based Swarm	Rule-Based Coordination
Collision Rate	3.2%	8.7%
Group Convergence Time	4.6 s	7.3 s
Avg. Path Optimality Score (0–1)	0.92	0.78
Adaptation to Moving Hazards	High	Moderate

Table 6: CAPQF Routing Impact Under Varying Densities

Agent Density	CAPQF Avg. Evac Time	Dijkstra Avg. Evac Time	CAPQF Survival Rate
100 agents (Low)	41.2 s	62.5 s	98.3%
300 agents (Medium)	55.8 s	79.4 s	93.1%
500 agents (High)	72.6 s	103.9 s	88.7%

THEXIT system is a notable improvement in intelligent evacuation techniques. The integration of CAPQF and PSO demonstrated superior performance in evacuation time, hazard avoidance and congestion reduction when compared to classic algorithms (i.e. Dijkstra) and appears effective relative to real-world disaster situations. Its flexibility,

decentralized form and real-time responsiveness give it a robust alternative for large-scale emergencies and it is proposed as a vital feature of future smart infrastructure design and disaster response.

V. CONCLUSION

By leveraging the capabilities of the Combined Adaptive Probabilistic Queue Function (CAPQF) and Particle Swarm Optimization (PSO), HEXIT is a major innovation in intelligent evacuation systems. It outperformed traditional algorithms like Dijkstra's in real-time adapted evacuation simulations performed in diverse virtual environments, from high-rises to busy public spaces, across multiple important evaluation metrics; these included reduced evacuation time, agent survival rate, hazard avoidance efficiency, and congestion decrease .The adaptable, decentralized nature of HEXIT, which performs effectively in diverse operational densities, means that it has practical applications in modern urban architecture and mass gathering events. Additionally, by simulating recent, real-world disaster scenarios (the 9/11 attacks or Mina), HEXIT not only demonstrated its superior technical capabilities, but also a capacity for life-saving impacts. Operational qualities, including swarm intelligence-based coordination that mitigated bottlenecks, added guarantees of evacuee safety while utilizing CAPQF to ensure optimal pathing decisions, even when some situational data was missing. The physical design, equipped with successful tests of its robust framework built with ROS2, Gazebo, and PyBullet, demonstrates the practicality of adding sensor-integration, decentralized into action. HEXIT further distances itself from an inflexible, centralized system towards a more adaptive, sensor-based system and identifies the potential for continued evolution towards smart cities, including the exploration of IoT-inspired safety infrastructure used in safe and prepared disaster protocols. it represents an essential innovation for improving public safety during emergencies.

VI. REFERENCE

[1] Lucas, T. A., & Tsai, J. P. (2015). A Particle Swarm Optimization Model of Emergency Airplane Evacuations with Emotion. *ResearchGate*.

[2] Al-Sultan, K., & Al-Fawzan, M. A. (2024). Comparative Analysis of Bellman-Ford and Dijkstra's Algorithms for Optimal Evacuation Route Planning in Multi-Floor Buildings. *Taylor & Francis Online*.

[3] Bouanan, Y., & Bouanan, M. (2023). A Simulation Evacuation Framework for Effective Disaster Management. *ScienceDirect*.

[4] GlobeNewswire. (2025). Intelligent Evacuation Systems: Global Strategic Business Report 2024-2025-2030. *GlobeNewswire*.

[5] Zhu, J., Li, W., Li, H., Wu, Q., & Zhang, L. (2017). A Novel Swarm Intelligence Algorithm for the Evacuation Routing Optimization Problem. *The International Arab Journal of Information Technology*, 14(6), 880-889.

[6] Shuaib, M., & Al-Omari, A. (2013). Decentralized Evacuation Management. *ScienceDirect*.

[7] Li, Y., & Liu, J. (2022). Spatial Allocation Method of Evacuation Guiders in Urban Open Spaces Based on PSO Algorithm. *PubMed Central*.

[8] Wang, H., & Zhang, X. (2023). Path Optimization for Mass Emergency Evacuation Based on an Improved Evacuation Optimization Model. *ScienceDirect*.

[9] Ghouzali, S., & Elhajj, I. H. (2023). A Methodology Based on Dijkstra's Algorithm and Mathematical Modeling for Identifying Optimal Evacuation Strategies. *ScienceDirect*.

[10] Chen, K., & Wang, Y. (2014). A Simulation Environment for the Dynamic Evaluation of Disaster Responses. *PubMed Central*.

[11] Grand View Research. (2023). Intelligent Evacuation System Market Size Report, 2030. *Grand View Research*.

[12] Zhu, Y., & Liu, H. (2020). Congestion-Aware Evacuation Routing Using Augmented Reality. *YZhu.io*.

[13] Becker, B., & Nagel, K. (2017). Integrating Decentralized Indoor Evacuation with Information Sharing through Mobile Devices. *MDPI*.

[14] Chen, X., & Zhan, F. B. (2024). Agent-Based Simulation for Pedestrian Evacuation. *ScienceDirect*.

[15] Balboa, A., González-Villa, J., Cuesta, A., Abreu, O., & Alvear, D. (2020). Testing a Real-Time Intelligent Evacuation Guiding System for Complex Buildings. *ScienceDirect*.

[16] Yamashita, A., & Tanaka, K. (2015). Congestion-Aware Route Selection in Automatic Evacuation Guiding Based on Cooperation Between Evacuees and Their Mobile Nodes. *Kansai University*.